Multivariable solution implementation:

```
feature_columns = ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']
```

- These are the features used to train the model.
- Next we call the following function to get the processed data

```
d_r = get_data(
    COMPANY,
    feature_columns,
    save_data=True,
    split_by_date=False,
    start_date=TRAIN_START,
    end_date=TRAIN_END,
    seq_train_length=PREDICTION_DAYS,
    steps_to_predict=STEPS_TO_PREDICT
)
```

This function returns scaled data, the actual scalers, data split into test and train.

```
if (scale is True):
    column_scaler = {}
    for column in feature_columns:
        # using min max scaler from scikit learn, normalises everything between 0
and 1
        scaler = MinMaxScaler()
        print(column)
        #   saves scaled data to each column in feature_columns, reshapes the
data to be 2D as is a requirement for MinMaxScaler
        data_df[column] =
scaler.fit_transform(np.expand_dims(data_df[column].values, axis=1))
        #   saves the specific minMax instances used for each column in a dict
for later use, like descaling.
        column_scaler[column] = scaler
    # print(column_scaler)
    #     add the MinMaxScaler instances to the result returned to have all
useful data/tools in one place
    result['column_scaler'] = column_scaler
```

- The code above scales te data for all the feature_columns and returns the scaled data. The expand_dims method reshapes data as a 2D array here because the min max scaler expects 2D shape for the data.

```
num_layers = 5
units_per_layer = 100
layer_name = LSTM
num_time_steps = PREDICTION_DAYS
number_of_features = len(feature_columns)
activation = "linear"
loss = "mean_squared_error"
optimizer = "RMSprop"
metrics = "mean_squared_error"
```

```
STEPS_TO_PREDICT = 5
```

- These are the parameters used to compile the model. We will explain STEPS_TO_PREDICT in the second titled MULTI_STEP Prediction. Please note the model can work with multiple features(multivariable training), determined by number_of_features.

This concludes the section of multivariable solution. Next we will discuss the multi step solution.

Multi-step solution:

```
for i in range(1, steps_to_predict + 1):
   future_col_name = f'future_{i}'
   data_df[future_col_name] = data_df['Close'].shift(-i)
   future_columns.append(future_col_name)
```

- In the code above, we create columns which are dynamically named as future_1, future_2 etc till steps_to_predict+1. We are basically shifting the Close price of each column so that the model can be trained to predict prices from 1 to steps_to_predict.

```
entry_sequences = deque(maxlen=seq_train_length)

feature_and_date_data = data_df[feature_columns + ['date']].values

future_values = data_df[future_columns].values

for index in range(len(feature_and_date_data)):
   # Entry contains the feature and date data, target contains the future
values that we aim to predict. We call it entry cuz it holds info about a
specific time point.

   entry = feature_and_date_data[index]
   target = future_values[index]
   entry_sequences.append(entry)
   if len(entry_sequences) == seq_train_length:
      # target_date = data_df['Date'].values[index + steps_to_predict - 1]
      # Adjust the index to get the correct future date
      data_in_sequence.append([np.array(entry_sequences), target])
```

- The code above first extracts the numerical values stored in data_df[future_columns] like future_1, future_2, future_3 that were created in the previous code snippet using the shift function.
- For each iteration, the current row is stored in the entry variable and for each iteration in the target variable, future values are stored. The future values are upto steps_to_predict. For example if steps_to_predict is 5, then we will have future values as stock prices for one day ahead, two days ahead, three days ahead, four days ahead and 5 days ahead. See table below for example:

Date	Close Price

2024-01-01	**100**
2024-01-02	**102**
2024-01-03	**105**
2024-01-04	**110**
2024-01-05	**115**
2024-01-06	**120**
2024-01-07	**125**
2024-01-08	**130**

So, the future values for 2024-01-01 are: `[102, 105, 110, 115, 120]`

- Also, entry_sequences is a double ended sequence, in which older data is removed once it reaches a given length, in this case steps_to_train.
- Once entry_sequences reach the specified length, it is appended to a list called data_in_sequences as an numpy array along with the target(which has future values).

This data processing gives us data for which model can be trained to predict k > 1 steps into the future.

```
STEPS_TO_PREDICT = 5
```

- This variable determines how far ahead into the future the model predicts values.

```
dl_model.add(Dense(steps_to_predict, activation=activation))
```

- In the create_model(...) function, we set the output layer as follows:
- This the last layer of the neural network and since we want steps_to_predict output, we set the number of neuron in this layer equal to steps_to_predict.

Results:

Following are the results obtained on the test set for steps_to_predict = 5

Actual Time Step 1:
Average Difference: 6.92
Average Percentage Difference: 4.86%

Actual Time Step 2:
Average Difference: 6.68
Average Percentage Difference: 4.87%

Actual Time Step 3:
Average Difference: 7.52
Average Percentage Difference: 5.41%

Actual Time Step 4:
Average Difference: 8.28
Average Percentage Difference: 5.97%

Actual Time Step 5:
Average Difference: 8.04
Average Percentage Difference: 5.90%

Total Average Difference (Actual): 7.49
Total Average Percentage Difference (Actual): 5.40%

References:

- Brownlee, J. (2017). *4 Strategies for Multi-Step Time Series Forecasting*. [online] Machine Learning Mastery. Available at: https://machinelearningmastery.com/multi-step-time-series-forecasting/.
- Brownlee, J. (2020). Multistep Time Series Forecasting with LSTMs in Python [online] Machine Learning Mastery. Available at: Multistep Time Series Forecasting with LSTMs in Python - MachineLearningMastery.com

- GeeksforGeeks. (2024). Multivariate Time Series Forecasting with LSTMs in Keras. [online] Available at: https://www.geeksforgeeks.org/multivariate-time-series-forecasting-with-lstms-in-keras/.